

- Este teste consiste de 3 questões.
- Responda às questões numa folha de exame.
- Pode definir funções auxiliares e/ou usar funções pré-definidas no prelúdio-padrão de Haskell.

1. (35%) Escreva uma definição recursiva da função `rld :: [(Int, Char)] → String` que descomprime uma cadeia de caracteres usando “*run-length encoding*”: cada sequência de caracteres repetidos é representada por um par com o comprimento e o carater correspondente.

Exemplo: `rld [(3, 'a'), (2, 'b'), (1, 'c'), (4, 'b')] = "aaabbcbbbb"`.

2. (35%) Considere o seguinte tipo recursivo para representar proposições lógicas.

```
data Prop = Const Bool      -- constantes
          | Var Char        -- variáveis
          | Neg Prop         -- negação
          | Conj Prop Prop   -- conjunção
          | Disj Prop Prop   -- disjunção
```

Escreva uma definição recursiva da função `dual :: Prop -> Prop` que obtém o *dual* duma proposição, i.e. a proposição que resulta de substituir todas as conjunções por disjunções (e vice-versa) e as constantes *True* por *False* (e vice-versa); as variáveis e negações são inalteradas. Exemplos:

```
dual (Neg (Var 'a'))
= Neg (Var 'a')

dual (Disj (Var 'x') (Neg (Var 'x')))
= Conj (Var 'x') (Neg (Var 'x'))

dual (Conj (Var 'a') (Disj (Var 'b') (Const False)))
= Disj (Var 'a') (Conj (Var 'b') (Const True))
```

3. (30%) Usando indução sobre listas, mostre a seguinte propriedade:

```
map f (xs++ys) = map f xs ++ map f ys
```

Sugestões: comece por escrever definições recursivas das funções `map` e `++` e use indução sobre `xs`.